중앙대 X 서강대

# 앵날다쏘

6조 앙바트로스

# Contents


01 결측치 처리

02 Feature Engineering

03 EDA

04 Modeling

05 Insight

06 Conclusion




# 01 결측치 처리

```
train.isnull().sum()/len(train) * 100
```

```
FlightDate                        0.000000
Origin                            0.000000
Dest                              0.000000
Cancelled                         0.000000
Diverted                          0.000000
CRSDepTime                        0.000000
DepTime                           2.578169
DepDelayMinutes                   2.582274
DepDelay                          2.582274
ArrTime                           2.683512
ArrDelayMinutes                   2.898303
AirTime                           2.915404
CRSElapsedTime                    0.000000
ActualElapsedTime                 2.895567
Distance                          0.000000
Year                              0.000000
Quarter                           0.000000
Month                             0.000000
DayofMonth                        0.000000
DayOfWeek                         0.000000
Marketing_Airline_Network         0.000000
IATA_Code_Marketing_Airline       0.000000
Flight_Number_Marketing_Airline   0.000000
Tail_Number                       0.919358
OriginCityName                    0.000000
OriginState                       0.000000
DestCityName                      0.000000
DestState                         0.000000
DepDel15                          2.582274
CRSArrTime                        0.000000
ArrDelay                          2.898303
ArrDel15                          2.898303
DistanceGroup                     0.000000
dtype: float64
```

```
train[train['Cancelled']==True].ArrDel15.value_counts()
```

```
Series([], Name: count, dtype: int64)
```

```
train[train['Diverted']==True].ArrDel15.value_counts()
```

```
Series([], Name: count, dtype: int64)
```

**결항/ 회항 True -> 제거**

# 01 결측치 처리

```
train.isnull().sum()
```

```
FlightDate                          0
Origin                              0
Dest                                0
CRSDepTime                          0
DepTime                             0
DepDelayMinutes                     4
DepDelay                            4
ArrTime                             0
ArrDelayMinutes                     4
AirTime                            29
CRSElapsedTime                      0
ActualElapsedTime                   0
Distance                            0
Year                                0
Quarter                             0
Month                               0
DayofMonth                          0
DayOfWeek                           0
Marketing_Airline_Network           0
IATA_Code_Marketing_Airline         0
Flight_Number_Marketing_Airline     0
Tail_Number                         0
OriginCityName                      0
OriginState                         0
DestCityName                        0
DestState                           0
DepDel15                            4
CRSArrTime                          0
ArrDelay                            4
ArrDel15                            4
DistanceGroup                       0
dtype: int64
```

남은 극소수 결측치도 drop

## [제거 후 결과]

```
df = train.copy()

print(f"전처리 전 행 수: {len(df):,}")

# 1. 결항 또는 회항된 행 완전 삭제
df = df[~ (df['Cancelled'] == 1) & ~ (df['Diverted'] == 1)].copy()

# 2. 이제 필요 없어진 두 컬럼 삭제
df = df.drop(columns=['Cancelled', 'Diverted'])

print(f"결항/회항 제거 후 행 수: {len(df):,}")
print(f"제거된 행 수: {len(train) - len(df):,} ({(len(train) - len(df))/len(train)*100:.2f}%)")
```

```
전처리 전 행 수: 146,189
결항/회항 제거 후 행 수: 141,956
제거된 행 수: 4,233 (2.90%)
```

# 02 피처 엔지니어링

## [시간대 DepHour]

```
#FlightDate를 날짜형으로 변환
train['FlightDate'] = pd.to_datetime(train['FlightDate'])
test['FlightDate']  = pd.to_datetime(test['FlightDate'])
```

```
# 시간대 피처

#예정 출발 시각(hhmm)을 시간만 추출
train['DepHour'] = train['CRSDepTime'] // 100
test['DepHour']  = test['CRSDepTime'] // 100

#아침 러시아워 (06:00~09:59 출발) → 1, 아니면 0
train['IsMorningRush'] = train['DepHour'].between(6, 9).astype(int)
test['IsMorningRush']  = test['DepHour'].between(6, 9).astype(int)

#저녁 러시아워 (17:00~20:59 출발) → 1, 아니면 0
train['IsEveningRush'] = train['DepHour'].between(17, 20).astype(int)
test['IsEveningRush']  = test['DepHour'].between(17, 20).astype(int)
```

1. 파생변수

- IsMorningRush: 6시-9시
- IsEveningRush: 17시-19시

```
time_features = ['DepHour', 'IsMorningRush', 'IsEveningRush', 'ArrDel15']

# 상관관계 계산
corr_time = train[time_features].corr()['ArrDel15'].drop('ArrDel15')
corr_time = corr_time.abs().sort_values(ascending=False)

print("시간대 피처와 ArrDel15 상관관계 (절댓값 기준)")
print("="*55)
print(corr_time)
```

```
시간대 피처와 ArrDel15 상관관계 (절댓값 기준)
=======================================================
DepHour          0.126282
IsMorningRush    0.094491
IsEveningRush    0.088718
Name: ArrDel15, dtype: float64
```

2. ArrDel15 - 시간대 피처 상관관계

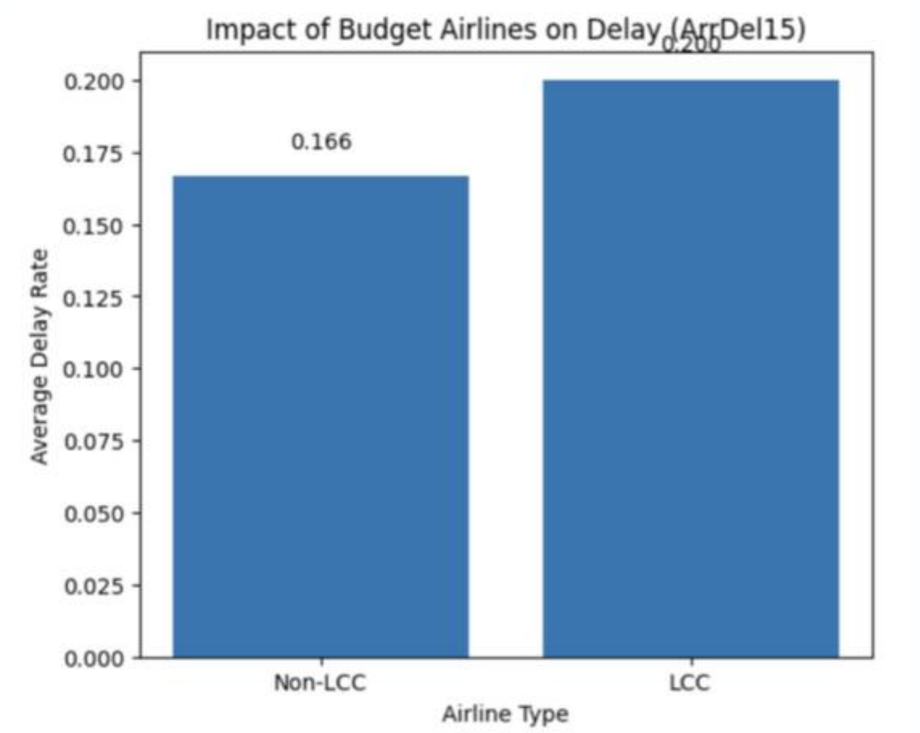
유의미한 상관관계

# 02 피처 엔지니어링

## [저가항공사 is_Budget_airline]

```
lcc_codes = [
    'WN',    # Southwest Airlines Co.
    'F9',    # Frontier Airlines Inc.
    'G4',    # Allegiant Air
    'NK',    # Spirit Air Lines
    'B6',    # JetBlue Airways
    'SY',    # Sun Country Airlines d/b/a MN Airlines
    'Y4',    # Volaris (멕시코 최대 LCC, 미국 노선 다수)
    'LJ',    # Jin Air (한국 LCC, 미국 노선 있음)
    'ZB',    # Monarch Airlines (과거 LCC)
    'Y2',    # Flyglobespan
    '0MQ',   # Air Choice One
    'L3',    # Lynx Aviation (Frontier 자회사)
    'PS'     # Pacific Southwest Airlines (역사적 LCC)
]
```

저가항공사 리스트

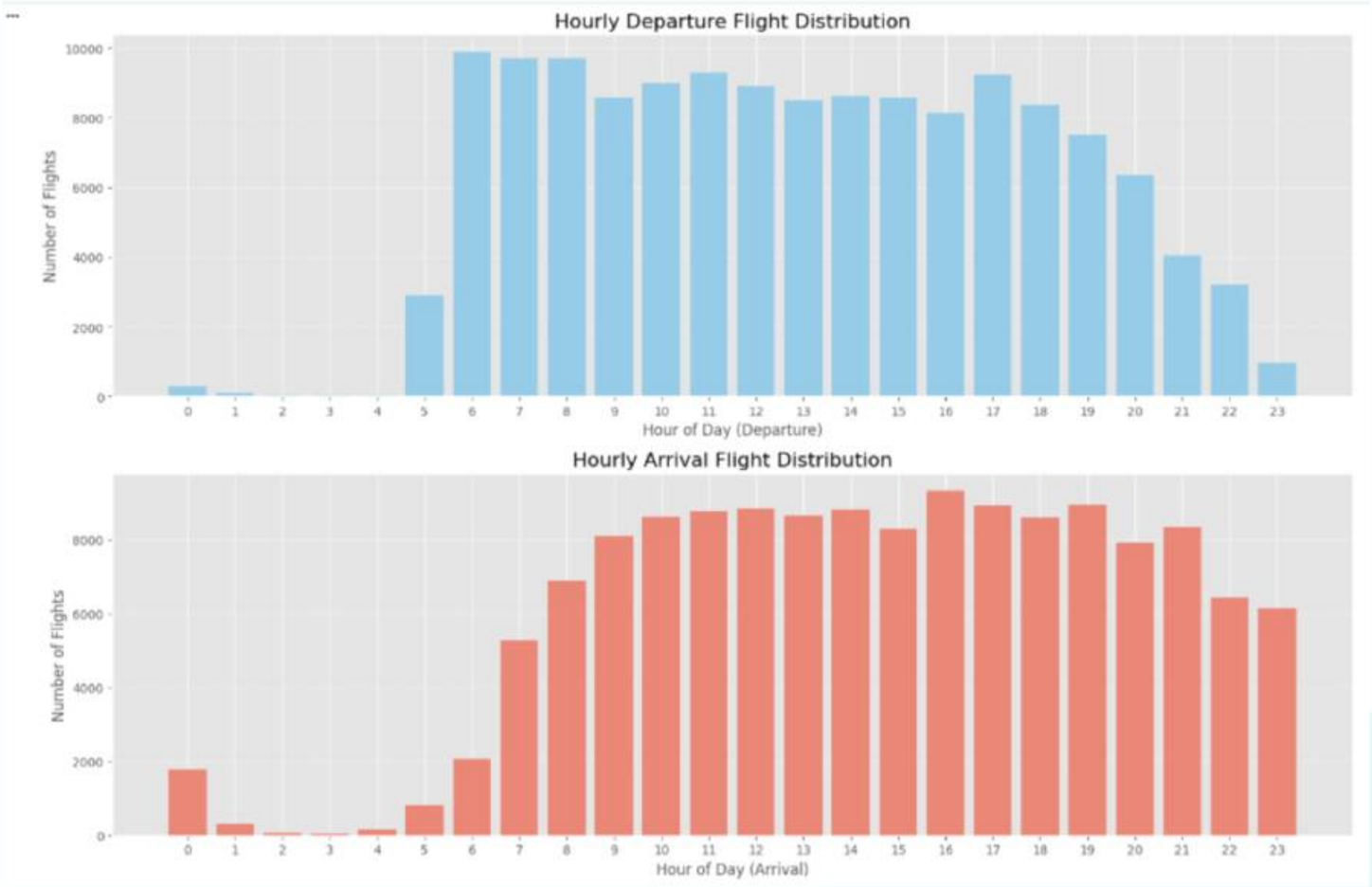



저가 항공사 비율이 지연에 미치는 영향

- 저가 항공사 O - 0.16
- 저가 항공사 X - 0.2

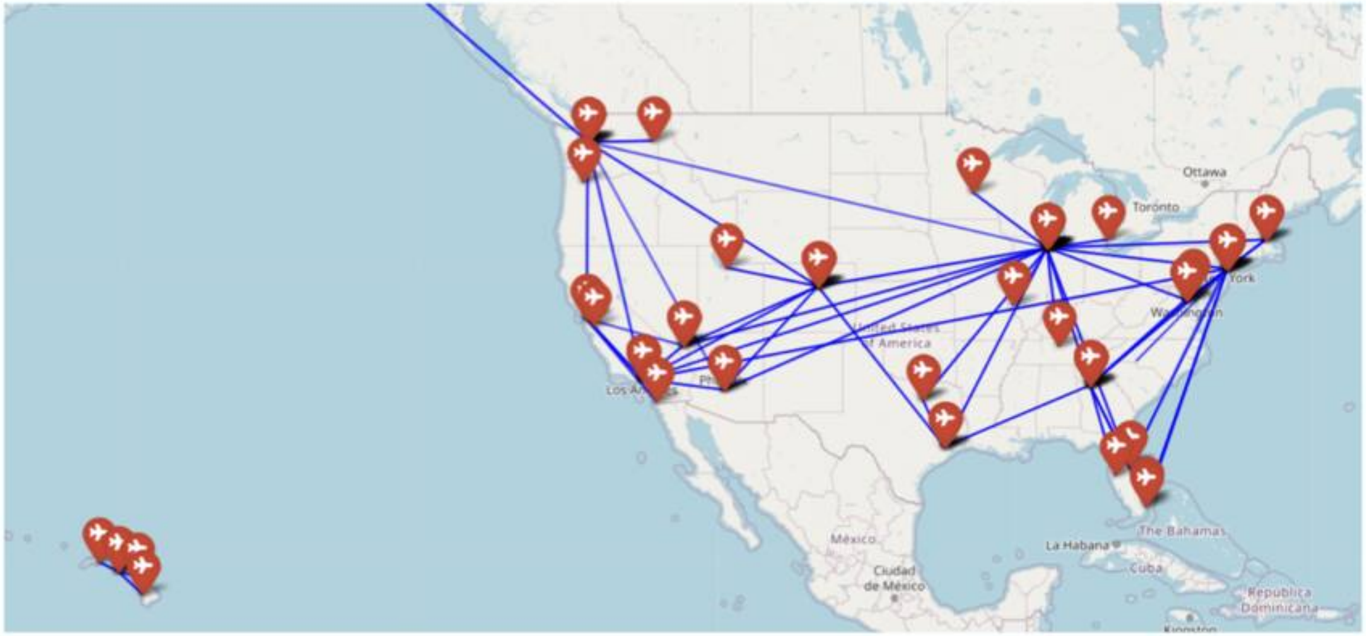
# 03 EDA

## [출발/도착 시간대별 운항 횟수]



# 03 EDA

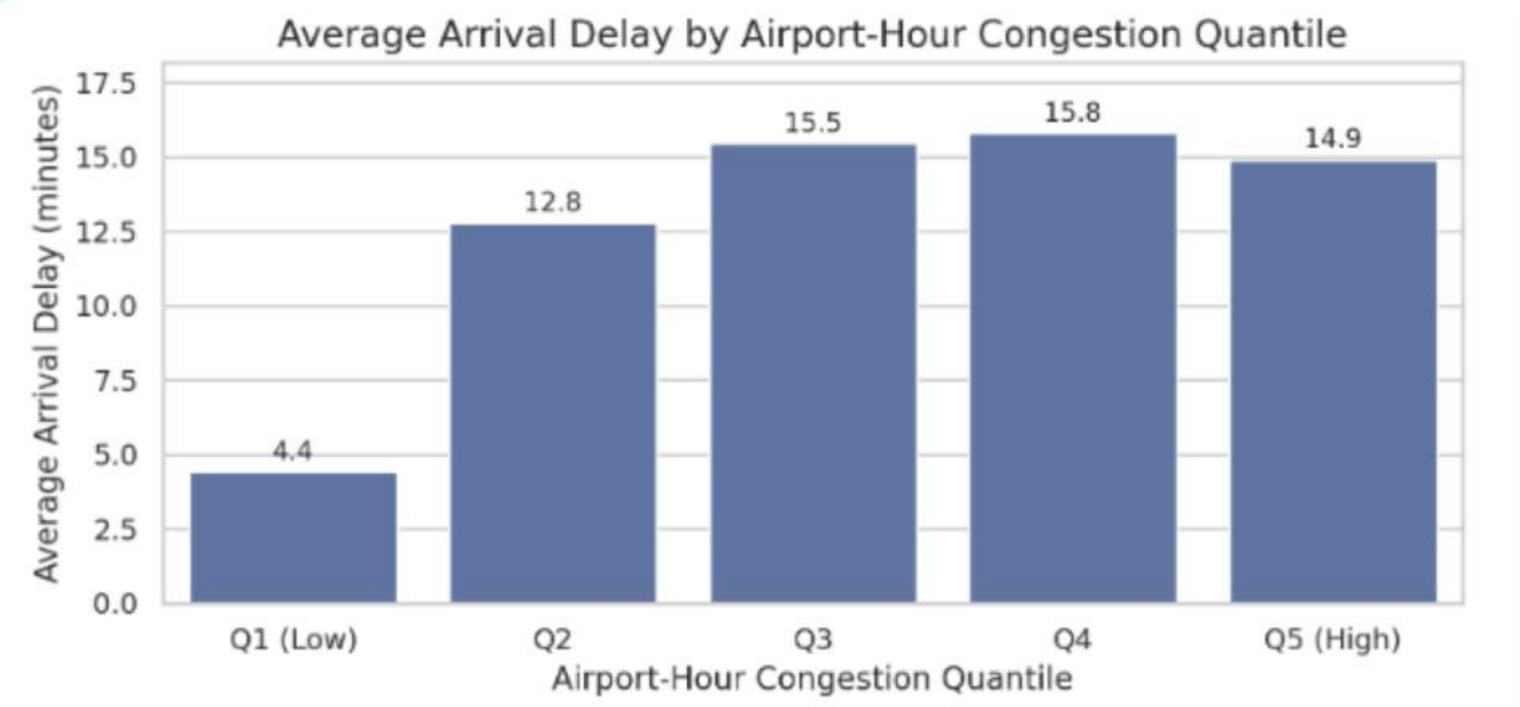
**[공항별 혼잡도 시각화]**




각 공항별로 출발/도착하는 비행기의 수가 다름

→ 공항의 혼잡도에 따라 지연시간에 영향가능성 有

# 03 EDA

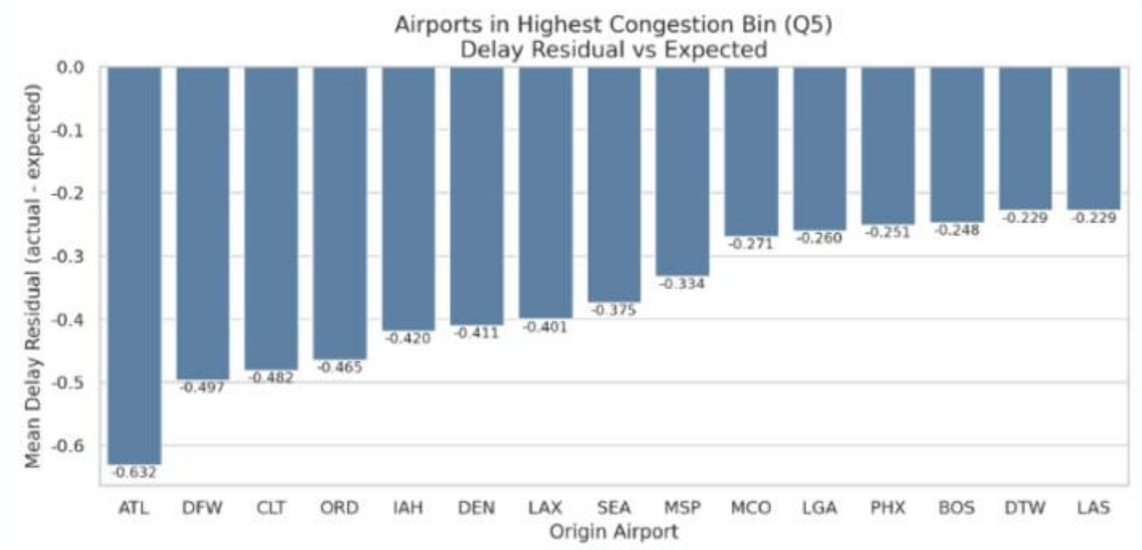



*혼잡도 : 각 공항에 대해서 시간대별로 출발&도착 건수의 합으로
한 공항에서 특정시간의 혼잡도를 점수화

혼잡할수록 평균 도착 지연시간 증가

But Q5에서 평균 지연시간 감소

# 03 EDA




Q5에 해당하는 항공사들과 각 항공사들의
예상 지연시간 - 실제 지연시간

ATL(애틀랜타 국제 공항) : 2024년 기준 세계에서 가장 승객수가 많은 공항

DFW(달라스포트워스 공항) : 북미 지역 공항 중 최우수 공항

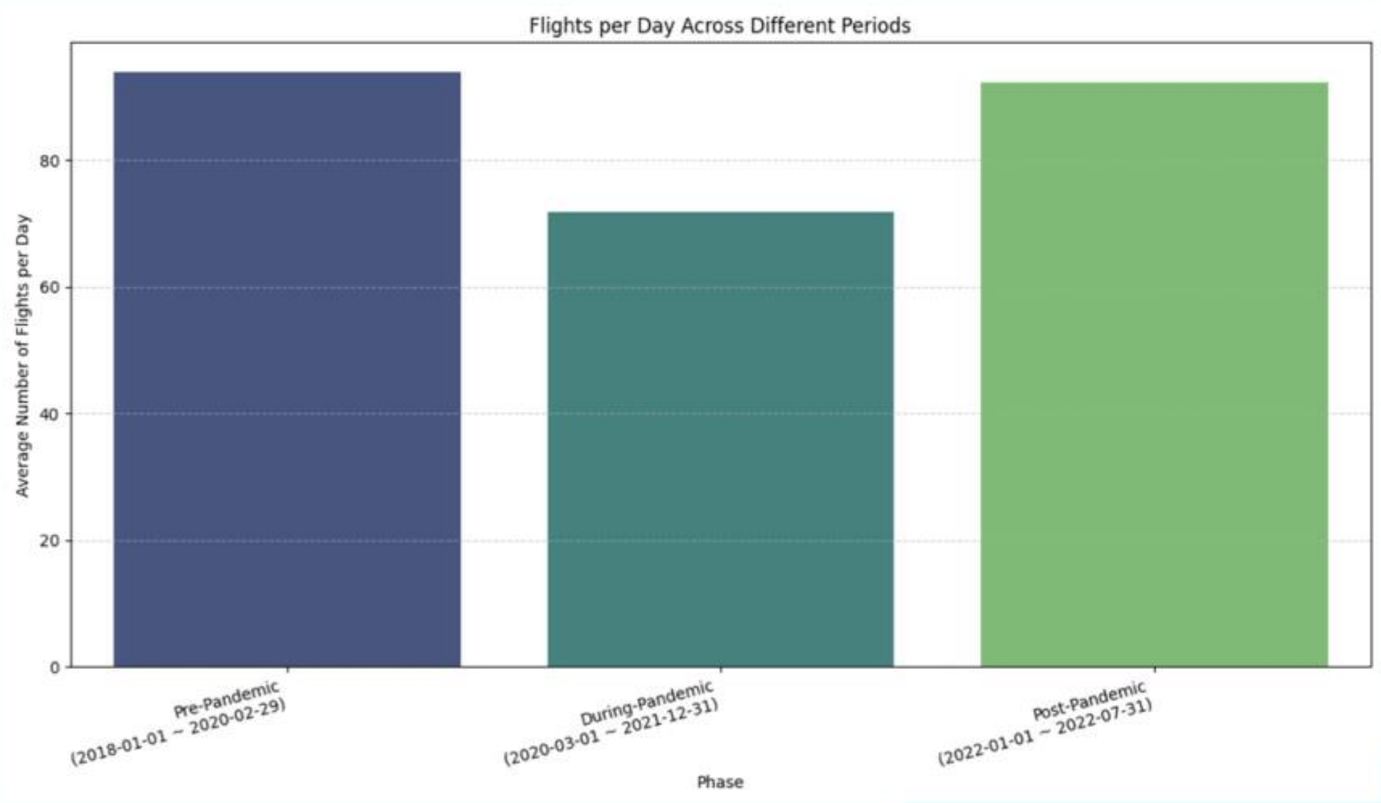
OLD(오헤어 국제 공항) : 미국 최우수 공항중 하나

LAX(로스앤젤레스 국제공항) : 서부에서 가장 큰 공항 중 하나

**→ Q5에 해당하는 공항들의 운영적 성숙도가 월등함**

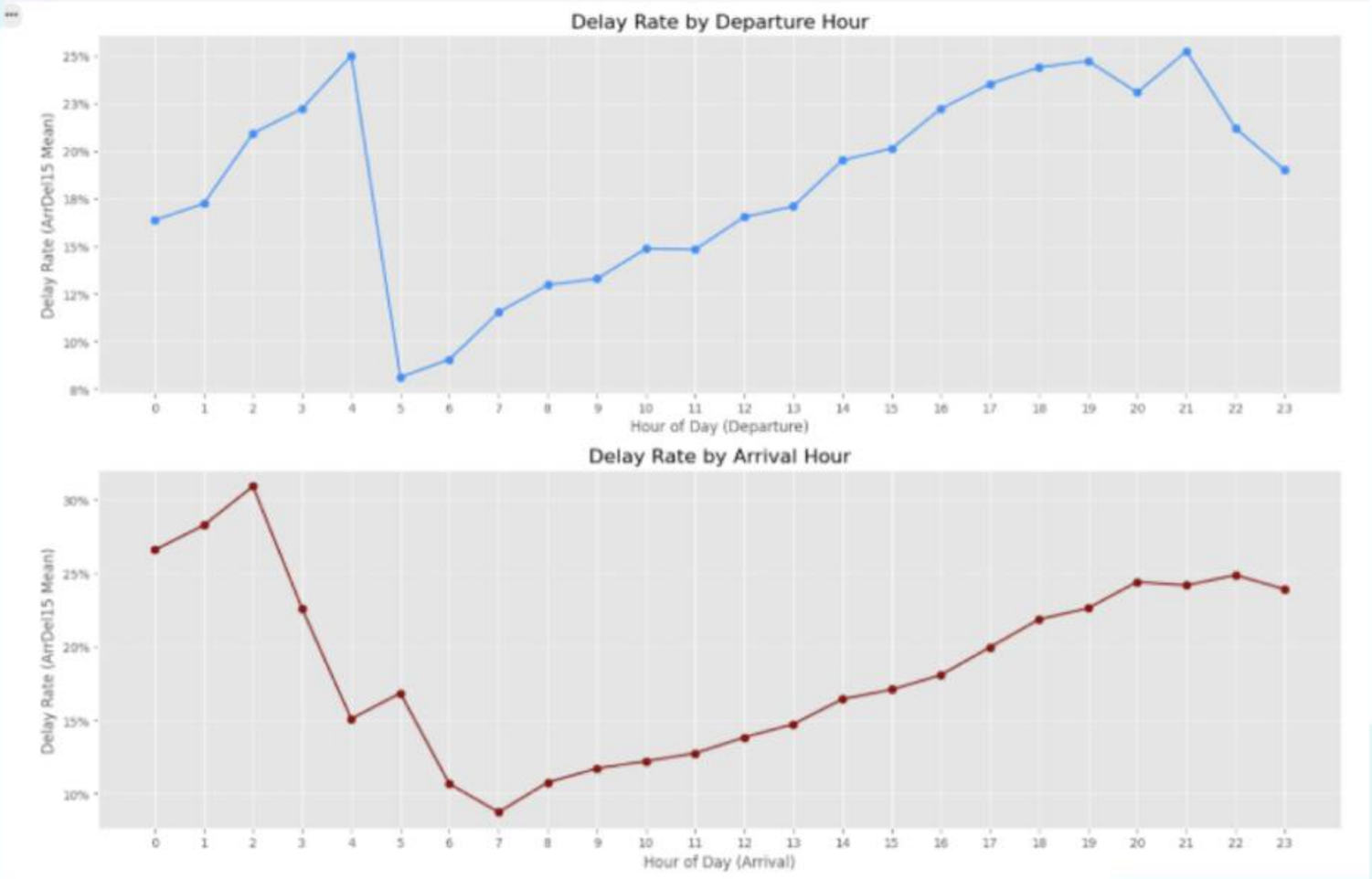
# 03 EDA

## [팬데믹 여부에 따른 시기별 항공편수]



# 03 EDA

## [출발/도착 시간대별 지연확률]




출발 시간이 늦어질 수록 지연률이 높아지는 경향

(도착 시간도 동일한 경향)

**항공 네트워크의 연쇄지연**
**-> 첫 비행편은 지연 확률이 낮지만, 시간이 지날수록 이전 지연이 누적되어 후속 편의 지연 확률이 증가**

# 04 Modeling

**선택 모델: LightGBM**
**선정 이유: 대용량 데이터셋에서 우수한 성능과 높은 효율성**
**(빠른 속도, 범주형 피처의 자동 처리, 불균형 데이터 처리 용이)**

```python
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

# 1. train에서 미래 정보(데이터 누설) 컬럼 완전 삭제
leakage_cols = ['DepTime', 'DepDelayMinutes', 'DepDelay', 'DepDel15',
                'ArrTime', 'ArrDelayMinutes', 'ArrDelay', 'AirTime',
                'ActualElapsedTime']

train = train.drop(columns=leakage_cols)
```

**#1 test set에 없는 컬럼 완전 삭제**

```python
target_col = 'ArrDel15'
X = train.drop(columns=[target_col])
y = train[target_col]

# 1. 불필요한 고유값 및 중복 정보를 가진 컬럼 삭제
cols_to_drop = [
    'FlightDate',                   # Year/Month 등으로 대체 가능
    'Tail_Number',                  # 고유값이 많아 모델에 노이즈를 줄 수 있음
    'Flight_Number_Marketing_Airline', # 고유값이 많음
    'OriginCityName',               # Origin과 중복 (도시 IATA 코드 사용 권장)
    'DestCityName',                 # Dest와 중복
    'OriginState',                  # Origin과 중복
    'DestState'                     # Dest와 중복
]
X.drop(columns=cols_to_drop, errors='ignore', inplace=True)
```

**#2 불필요한 고유값 및 중복 정보를 가진 컬럼 삭제**

# 04 Modeling

```python
# 2. 범주형 특성 식별 및 'category' 타입으로 변환 (LightGBM 최적화)
# 'object' 타입 컬럼을 찾음
categorical_features = X.select_dtypes(include='object').columns.tolist()
print(f"변환할 범주형 컬럼: {categorical_features}")
for col in categorical_features:
    # LightGBM의 native categorical feature 처리 기능을 활용하기 위해 'category' 타입으로 변환
    X[col] = X[col].astype('category')

# 3. 학습 및 검증 데이터셋 분리
X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42, # 시드 고정
    stratify=y       # 타겟 변수의 분포를 유지하며 분리
)
```

**#3 LightGBM 최적화를 위해 범주형 특성 식별 및 'category' 타입으로 변환**

# 04 Modeling

```python
# 4. LightGBM Dataset 생성
# LightGBM은 이 단계에서 'category' 타입을 인식하여 내부적으로 처리
lgb_train = lgb.Dataset(X_train, y_train, categorical_feature=categorical_features)
lgb_val = lgb.Dataset(X_val, y_val, reference=lgb_train, categorical_feature=categorical_features)


# 5. 파라미터 및 학습 진행 (콜백 함수를 사용하여 이전 오류 해결)
params = {
    'objective': 'binary',  # 타겟 변수가 0/1이므로 이진 분류로 설정
    'metric': 'binary_logloss',
    'boosting_type': 'gbdt',
    'n_estimators': 3000,
    'learning_rate': 0.05,
    'feature_fraction': 0.8,
    'bagging_fraction': 0.8,
    'bagging_freq': 1,
    'lambda_l1': 0.1,
    'lambda_l2': 0.1,
    'verbose': -1,          # 학습 중 메시지 출력 방지
    'n_jobs': -1,
    'seed': 42
}

# 콜백 정의 (early_stopping_rounds, verbose_eval 오류 해결)
callbacks = [
    early_stopping(stopping_rounds=100, verbose=True),
    log_evaluation(period=100)
]
```

#4 LightGBM Dataset 생성 및 파라미터 생성

```python
# 6. 학습 시작
print("\n========== LightGBM 학습 시작 ==========")

model = lgb.train(
    params,
    lgb_train,
    valid_sets=[lgb_train, lgb_val],
    num_boost_round=3000,
    callbacks=callbacks
)
print("\n========== LightGBM 학습 완료 ==========")
```

#5 학습 시작

# 04 Modeling

```
========================================
            모델 성능 평가 (Validation Set)
========================================
ROC AUC Score: 0.6571
Accuracy: 0.8238
Precision (지연 예측 정확도): 0.5000
Recall (실제 지연을 맞춘 비율): 0.0104
F1 Score: 0.0204

Confusion Matrix (오차 행렬):
[[23331    52]
 [ 4949    52]]
 [TN, FP]
 [FN, TP]
========================================
```

**#6 모델 평가**

05 Insight

## PageRank를 사용해서 각 공항별 특징을 분석해보자




전파되는 현상이 페이지링크 알고리즘과 유사

기본 구조

- node(page) : 각 공항
- edge : 노선 (공항 to 공항)

# Conditional Weighted PageRank

## 1. 알고리즘의 목표 (Objective)

"지연은 단순한 통계가 아니라, 시간대에 따라 역할이 변하는 네트워크 현상임을 증명한다."

- 각 공항에 '전파자', '누적자' 라는 두 가지 역할을 부여하고 각 능력치를 계산
  - 전파자 : 처음 지연을 네트워크에 전파하는 공항
  - 누적자 : 오후~저녁 중 혼잡도로 인해 지연을 누적 및 심화시키는 공항

## 2. 활용한 핵심 데이터 (Input Features)

1. Origin, Dest (노선 정보):
   - 그래프의 Node, Edge를 정의
2. CRSDepTime, CRSArrTime (시간 정보):
   - "아침 전파력" (6-10시 출발)와 "저녁 누적력" (18-22시 도착)을 계산하기 위해 항공편을 시간 기준으로 분리
3. ArrDel15 (지연 여부):
   - 각 엣지(노선)가 얼마나 위험한지를 측정하는 **'엣지 가중치(W)'**를 계산하는 데 사용했습니다.

# Conditional Weighted PageRank

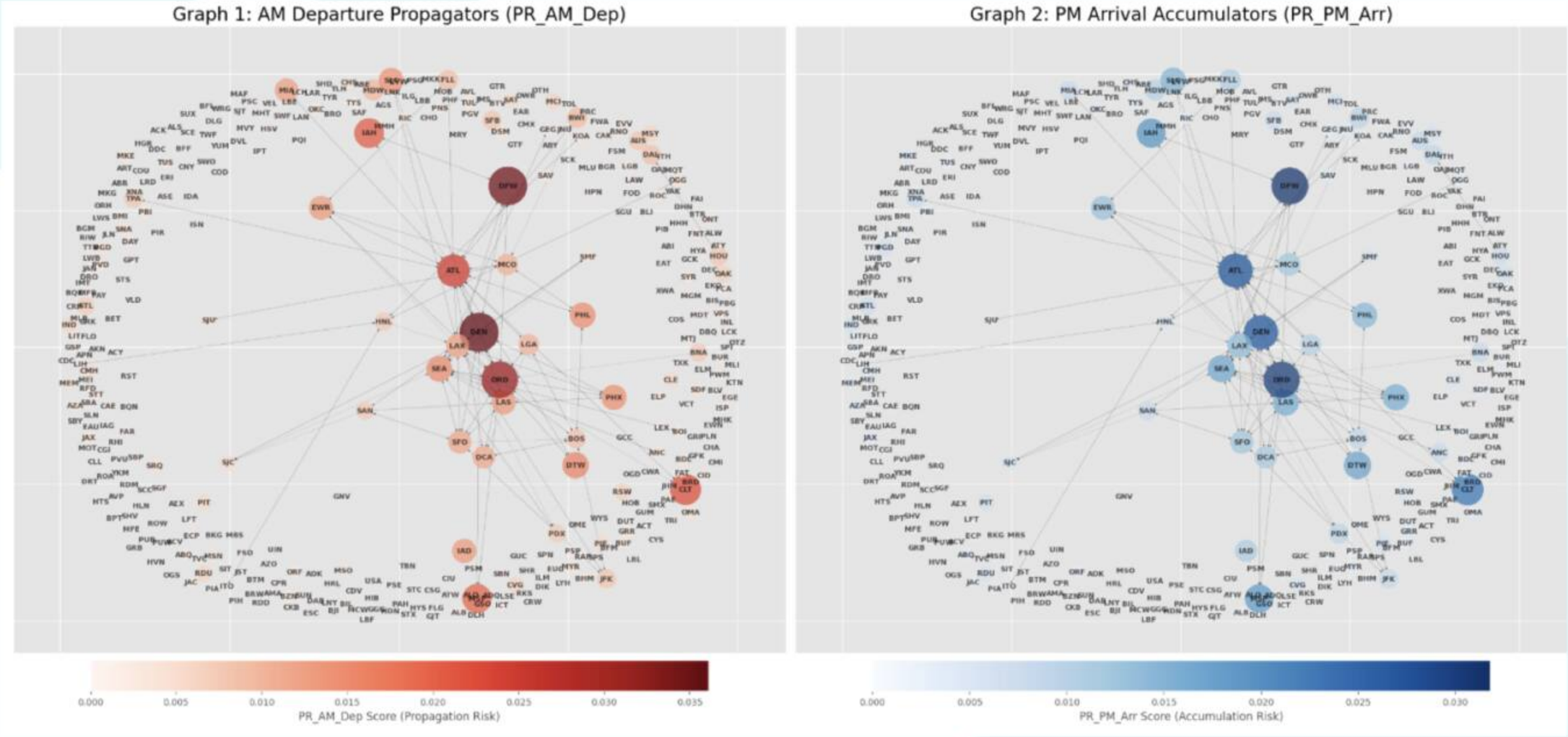
3. 핵심 수식: '엣지 가중치(W)'의 정의 (The Formula)

**베이지안 스무딩(Bayesian Smoothing)**

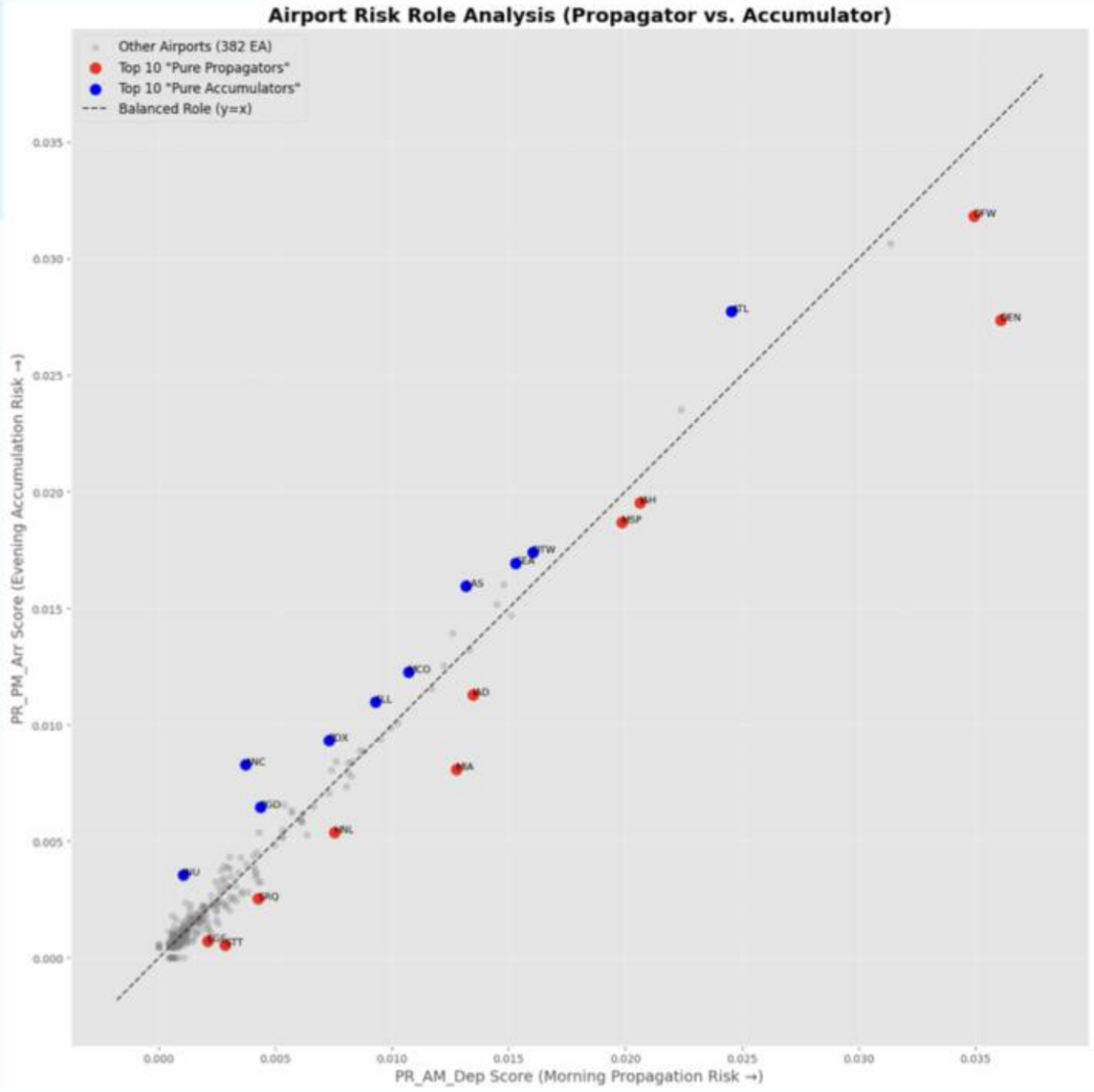
$$W = \text{Smoothed Rate} = \frac{(N_{\text{route}} \times R_{\text{route}}) + (C \times R_{\text{global}})}{N_{\text{route}} + C}$$

- $N_{\text{route}}$: 해당 노선의 **총 운항 횟수** ( `count` )
- $R_{\text{route}}$: 해당 노선의 **평균 지연율** ( `delay_rate` )
- $R_{\text{global}}$: **전체 데이터의 평균 지연율** ( `global_mean_delay` )
- $C$: **보정 계수** (코드에서 `50` 으로 설정). 운항 횟수가 $C$보다 적은 노선의 지연율을 $R_{\text{global}}$ 쪽으로 당겨와, 통계적 신뢰도를 보정합니다.

# PageRank 결과


Graph 1: AM Departure Propagators (PR_AM_Dep)
0.000
0.005
0.010
0.015
0.020
0.025
0.030
0.035
PR_AM_Dep Score (Propagation Risk)
Graph 2: PM Arrival Accumulators (PR_PM_Arr)
0.000
0.005
0.010
0.015
0.020
0.025
0.030
PR_PM_Arr Score (Accumulation Risk)

# PageRank 결과



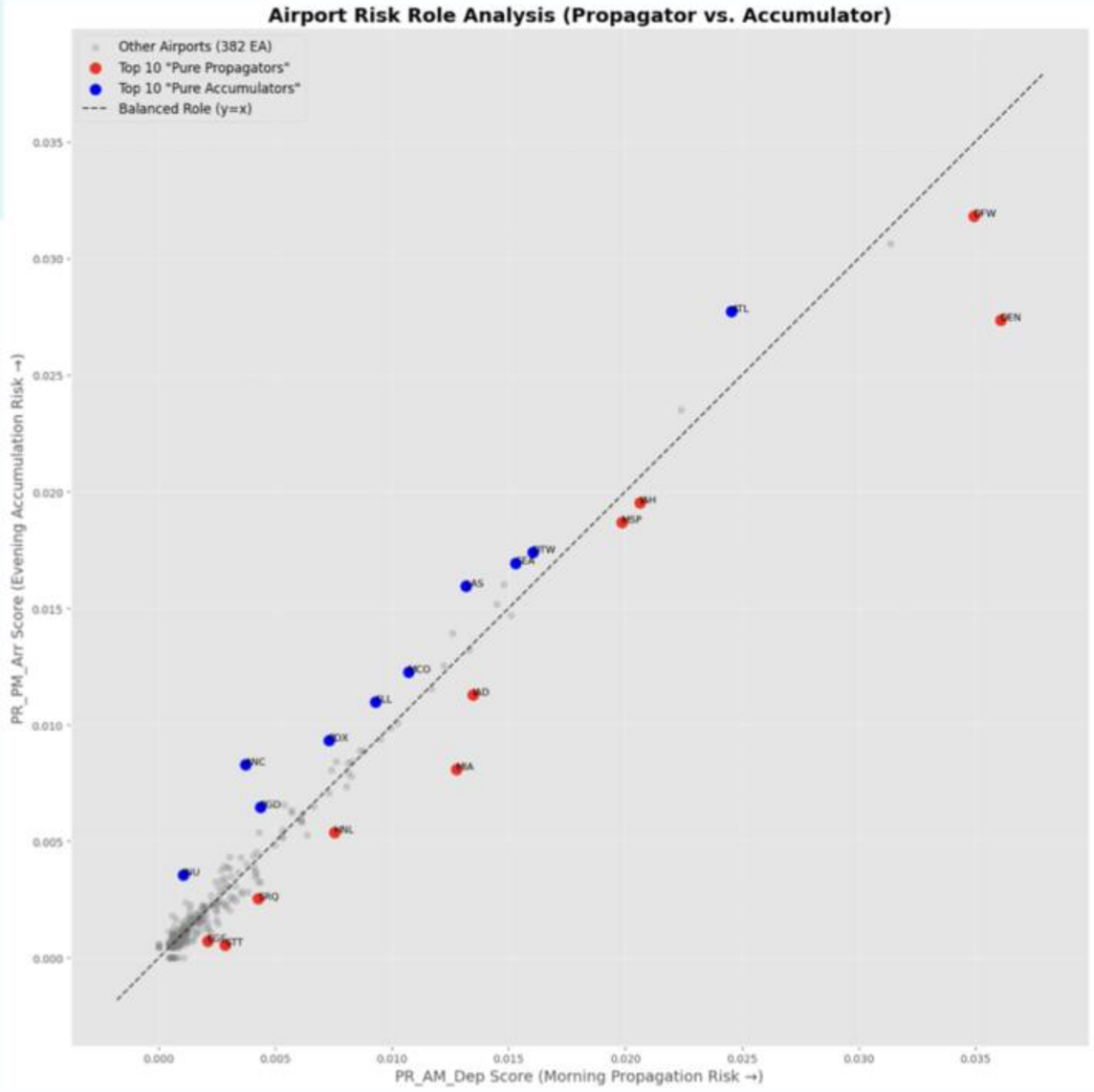

## 주목해볼만한 점

1. 대부분은 전파력과 누적력이 비슷한 경향을 보임
   - 혼잡한 공항은 지연 전파와 누적 모두 영향이 크다.
   - 여유로운 공항은 지연 전파와 누적 모두 영향이 작다.

2. 대형 공항은 전파력과 누적력이 모두 높은 경향을 보임

3. 전파력이 강한 / 누적력이 강한 공항이 존재함
   - 전파력이 누적력에 비해 강한 공항
     - DEN, DFW, MIA (대형 허브) : 항공편을 중앙에서 흩뿌리는 역할
     - STT, EGE, HNL (레저/관광지) : 절대적 혼잡도는 낮아보이지만, 휴양 후 돌아가는 인원들로 지연이 시작되는 것으로 추정 됨 (많은 짐, 가족 단위 여행객)

# PageRank 결과 (가로축 : 전파력 / 세로축 : 누적력




## 주목해볼만한 점

3. 전파력이 강한 / 누적력이 강한 공항이 존재함

- 전파력이 누적력에 비해 강한 공항
  - DEN, DFW, MIA (대형 허브) : 항공편을 중앙에서 흩뿌리는 역할
  - STT, EGE, HNL (레저/관광지) : 절대적 혼잡도는 낮아보이지만, 휴양 후 돌아가는 인원들로 지연이 시작되는 것으로 추정 됨 (많은 짐, 가족 단위 여행객)

- 누적력이 전파력에 비해 강한 공항
  - 동부지역 시차 영향으로 오후 도착 편이 많음
  - 최종 목적지

# 해결방안

**1. 📈 운영 효율성 개선**

전파자 공항 (오전 출발 집중): 오전 피크 시간의 혼잡을 줄이기 위해 체크인/보안 검색대 운영 인력을 오전 시간대에 집중 배치하고, 이른 아침/늦은 밤 시간대 항공편에 대한 인센티브를 제공하여 트래픽을 분산합니다.

누적자 공항 (오후 도착 집중): 오후 피크 시간의 수하물 처리 및 지상 조업 인력을 강화하고, 대중교통 및 셔틀 버스 배차를 오후 시간대에 늘려 승객의 최종 이동(Last Mile) 병목 현상을 완화해야 합니다.

**2. ✈️ 노선 조정 및 전략**

전파자 공항: 오후 출발 항공편 수를 전략적으로 늘려 균형 있는 스케줄을 유도하여, 오전에 발생한 트래픽 불균형을 해소합니다.

누적자 공항: 오전 도착 항공편을 유치하거나, 다른 공항으로의 환승 연결을 강화하여 단순 누적(도착)이 아닌 재분배(환승) 기능도 수행하도록 유도합니다.

# 06 Conclusion

공항 혼잡도가 지연확률에 영향을 미치지만,
공항 및 항공사의 운영성숙도에 따라 지연이 적어지기도 함.
지연에 대응하기 위해서는 시스템구축에 노력하기를 권장함.

# 수고하셨습니다

특히 우리팀...;ㅁ;